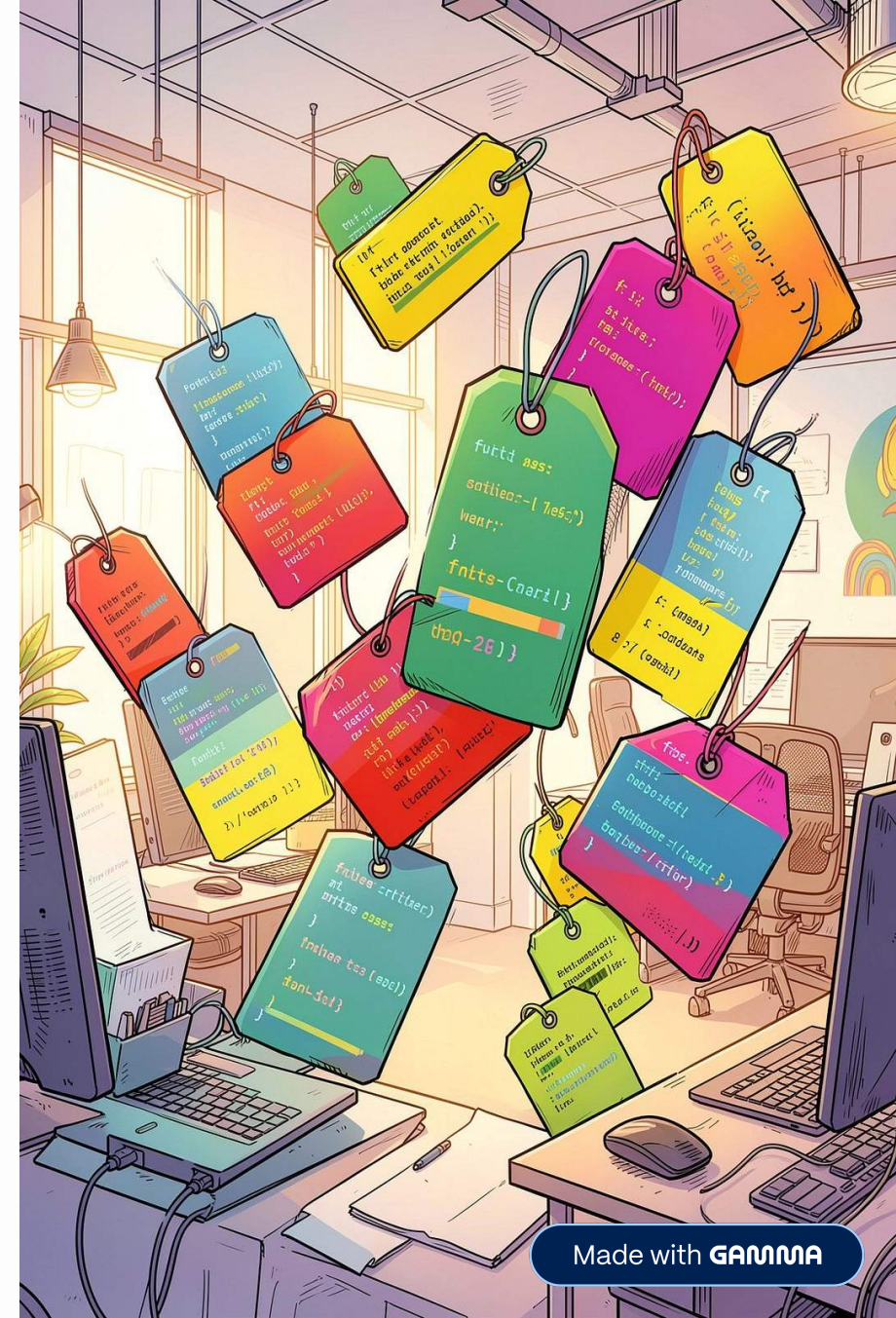


Lenguajes de Marcas: XML y HTML

Una introducción visual para estudiantes que comienzan desde cero.



¿Qué es un Lenguaje de Marcas?

Sistema de Etiquetas

Un lenguaje de marcas usa **tags** para etiquetar y organizar información, dándole estructura y significado al contenido.

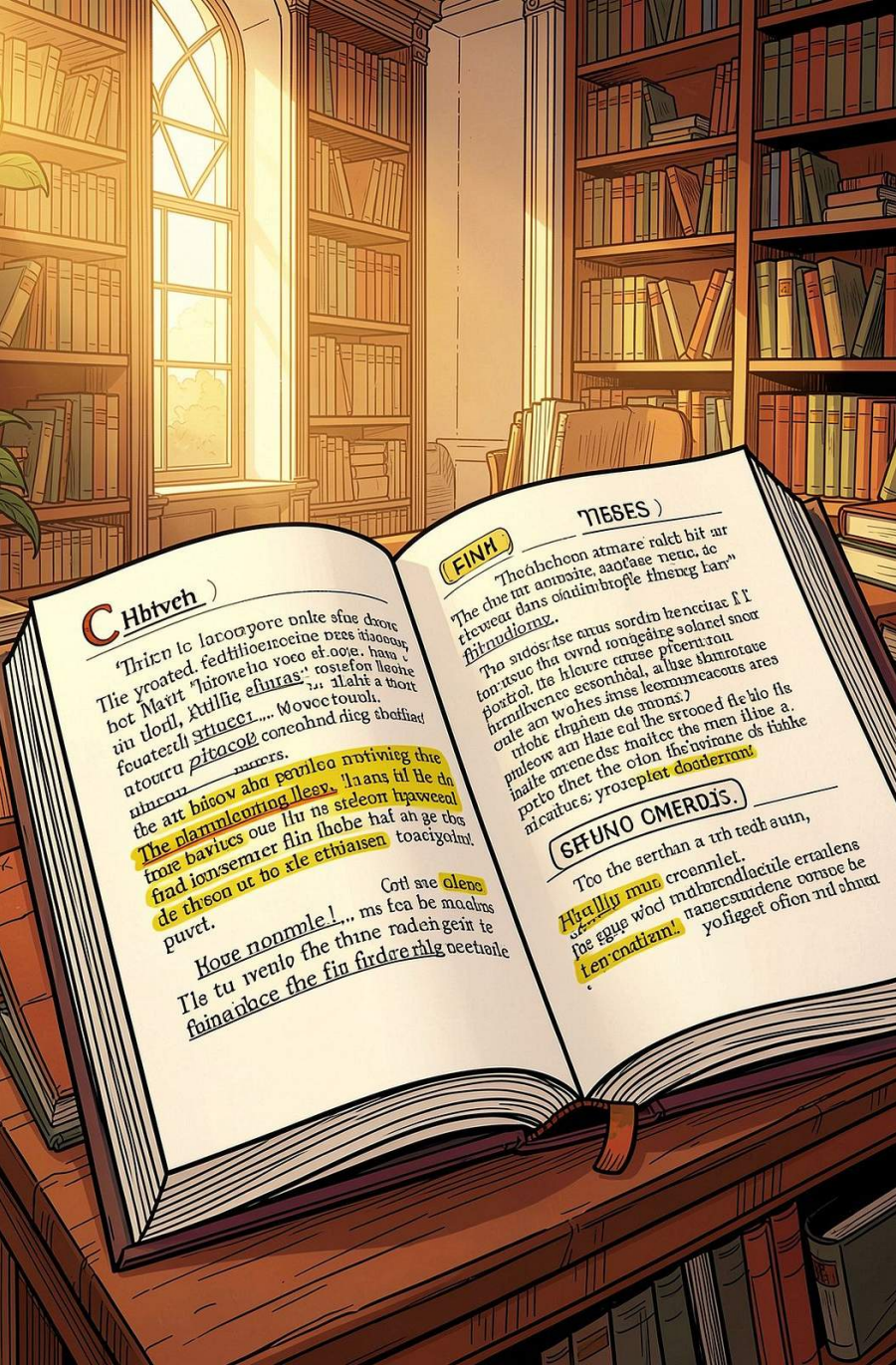
Las Marcas Envuelven el Contenido

Cada tag rodea un fragmento de texto para indicar qué tipo de información es. Los tags van en pares: uno de apertura y uno de cierre.

Ejemplo Sencillo

```
<titulo>Mi Primer Documento</titulo>
```

El tag **<titulo>** le dice al sistema: "esto es un título".





HTML

HTML: La Web que Ves

HTML es el lenguaje que da forma visual a las páginas web. Le dice al navegador cómo mostrar el contenido al usuario.

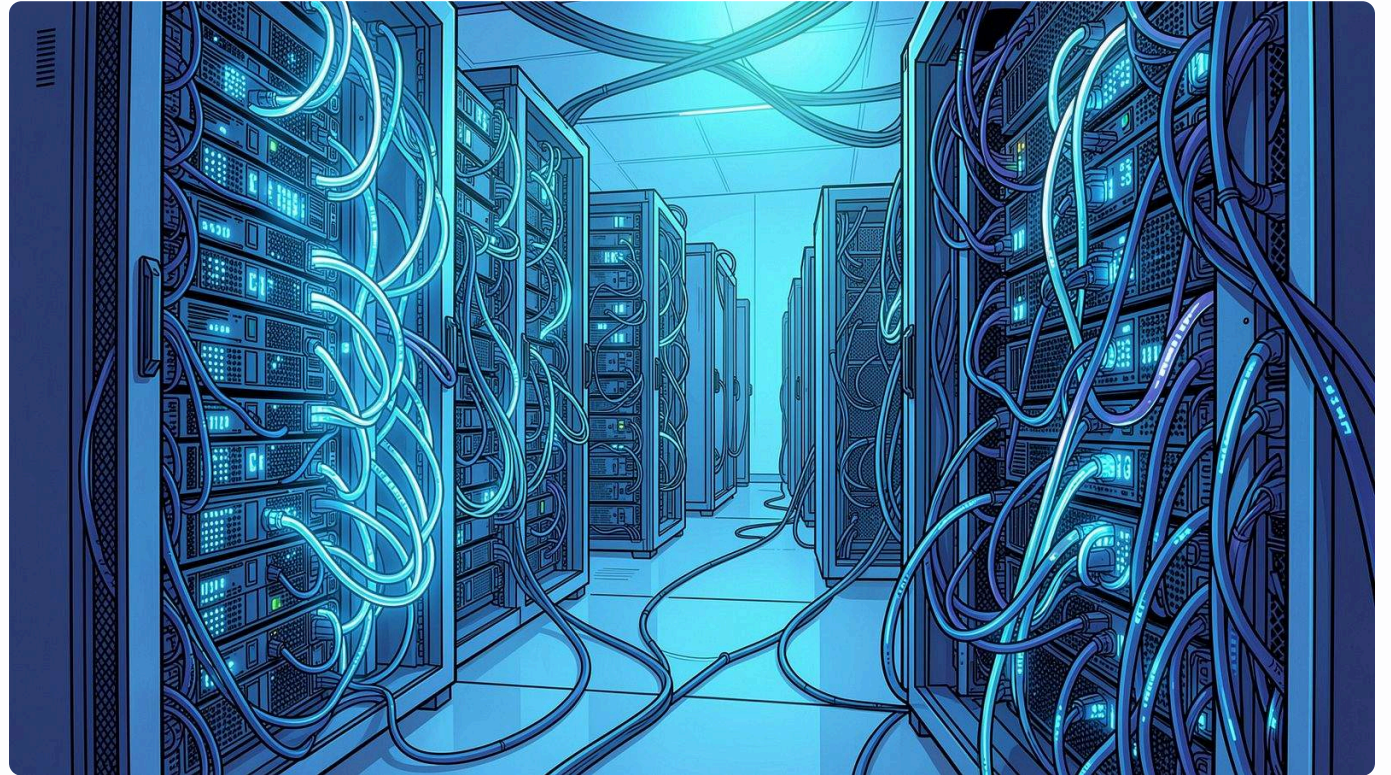
- Crea páginas web con **estructura y formato visual**
- Define cómo se ve el contenido en el navegador
- `<h1>` para títulos grandes
- `<p>` para párrafos de texto

XML: Datos Organizados

XML está diseñado para **almacenar y transportar datos** de forma clara y estructurada, no para mostrarlos visualmente.

- Organiza datos de forma estructurada y legible
- No define apariencia visual — solo **significado**
- Comprensible tanto para máquinas como para personas

❏ `<precio>29.99</precio>` —
Cualquiera entiende que ese número es un precio.



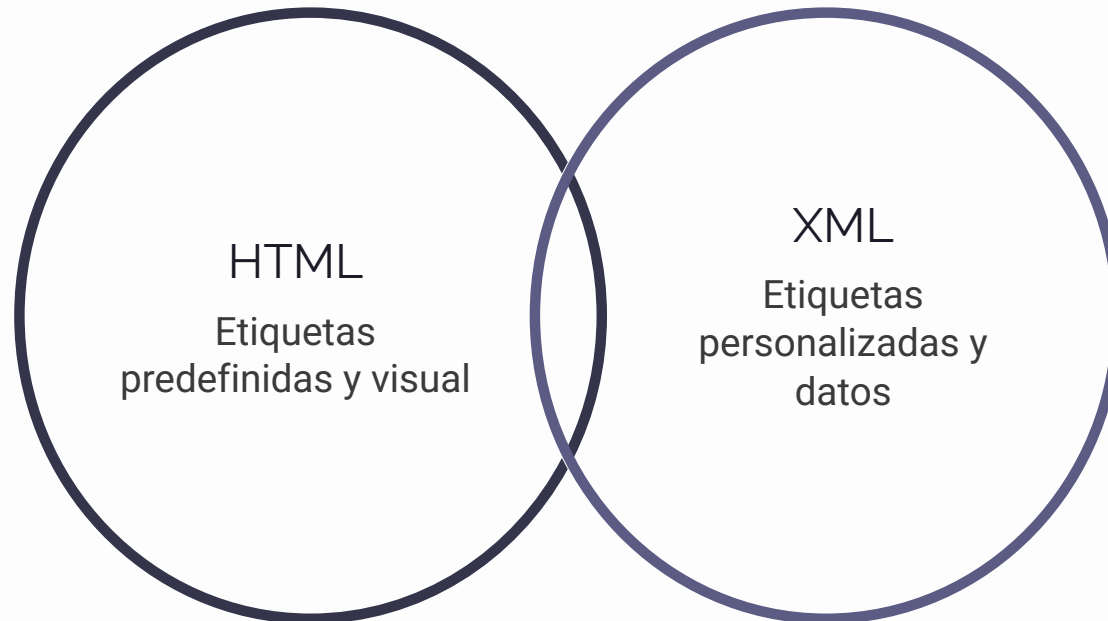
HTML vs XML: La Diferencia Clave

HTML — ¿Cómo se ve?

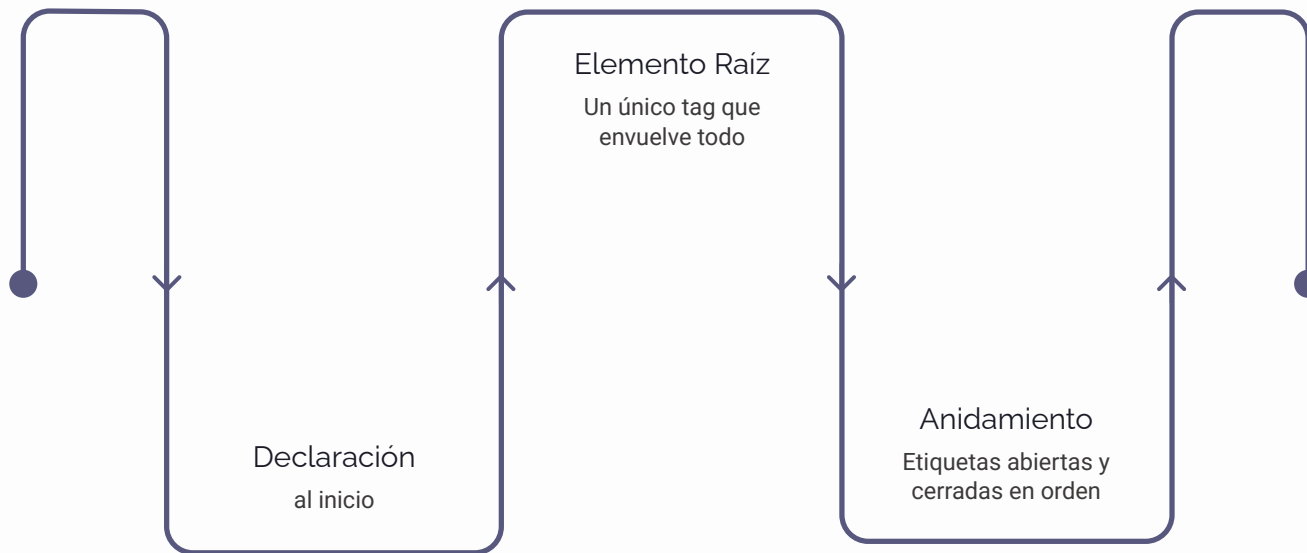
Enfoque **visual**. Usa tags predefinidos como `<h1>`, `<p>`, `<img>` para dar formato a páginas web en el navegador.

XML — ¿Qué significa?

Enfoque en **datos**. Permite crear tus propios tags personalizados. El significado es lo más importante, no la apariencia.



Estructura Básica de XML



Las tres partes esenciales

1 Declaración

```
<?xml version="1.0"?>
```

Siempre va al inicio del documento.

2 Elemento Raíz

Todo el contenido debe estar dentro de **un único tag principal** que lo contenga todo.

3 Anidamiento Correcto

Los tags se abren y cierran **en el orden correcto**, siempre de adentro hacia afuera.

Ejemplo XML: Una Receta

Así se vería una receta de cocina expresada en XML. Cada elemento describe **qué es** el dato, no cómo mostrarlo:

```
<?xml version="1.0"?>
<receta>
  <nombre>Pasta
Carbonara</nombre>
  <tiempo>20
minutos</tiempo>
  <ingredientes>
    <item>Huevos</item>
    <item>Panceta</item>
    <item>Queso
Parmesano</item>
  </ingredientes>
</receta>
```

Tag raíz

`<receta>` contiene todo el documento.

Datos descriptivos

`<nombre>` y `<tiempo>` identifican cada dato claramente.

Anidamiento

`<item>` está dentro de `<ingredientes>`, que está dentro de `<receta>`.



Reglas Básicas de XML

XML es muy estricto. Si no se siguen estas reglas, el documento no funcionará correctamente.



Tags Siempre Cerrados

Todo tag abierto debe cerrarse.

`<nombre>Ana</nombre>` ✓

`<nombre>Ana` ✗



Sensible a Mayúsculas

XML distingue entre mayúsculas y minúsculas.

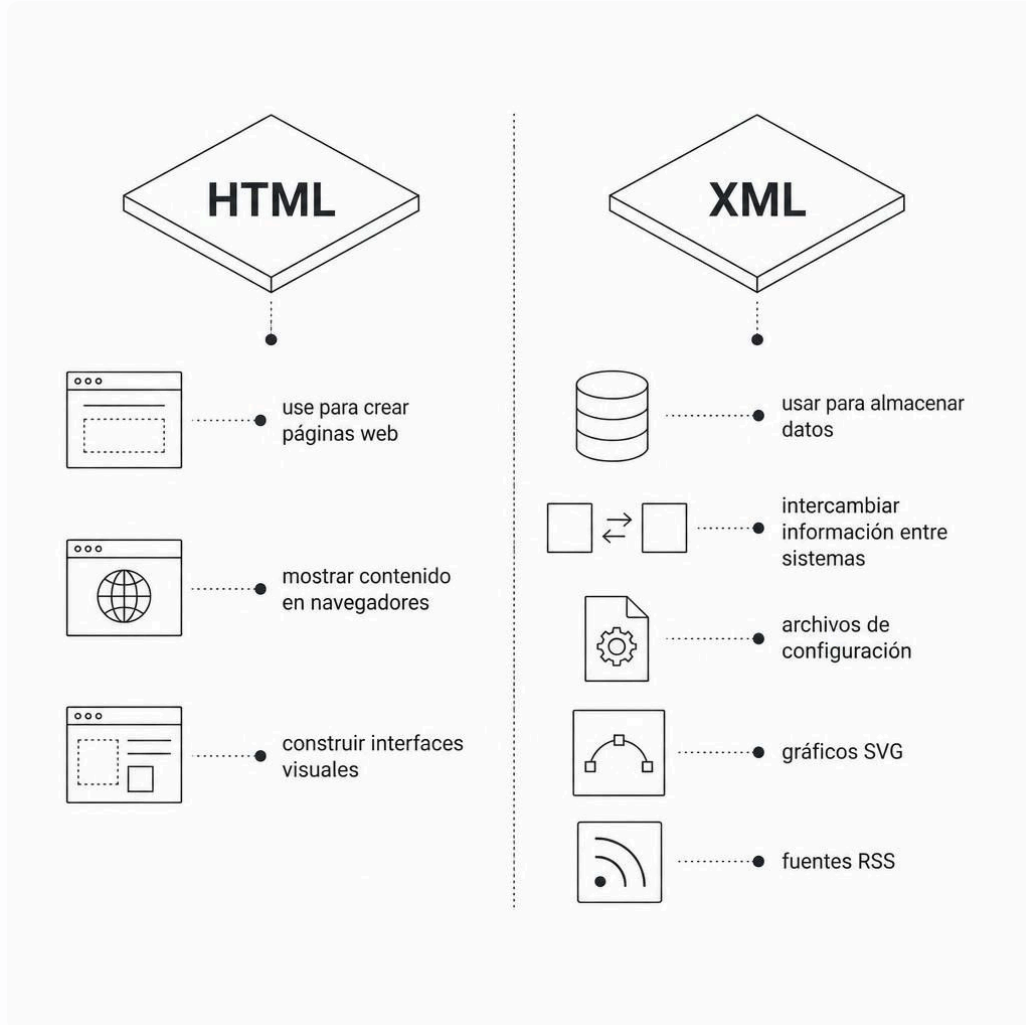
`<Nombre>` y `<nombre>` son tags **completamente diferentes**.



Un Solo Elemento Raíz

Debe existir **un único tag raíz** que envuelva todo el contenido del documento XML.

¿Cuándo Usar Cada Uno?



La regla de oro

→ Elige HTML

Cuando quieras **crear páginas web** y mostrar contenido visualmente en un navegador.

→ Elige XML

Cuando necesites **guardar o intercambiar datos** entre sistemas, aplicaciones o servicios.

→ Bonus

Muchas tecnologías modernas usan XML internamente: **SVG, RSS y archivos de configuración.**

DTD: Validar la Estructura de XML

Para que los documentos XML sean realmente útiles, deben seguir una estructura consistente. Aquí es donde entra en juego la DTD.

¿Qué es una DTD?

Una **Definición de Tipo de Documento** (DTD) es un conjunto de reglas que definen los elementos, atributos y relaciones permitidos dentro de un documento XML.

¿Para qué sirve?

Asegura que todos los documentos XML de un mismo tipo (ej. "recetas") tengan la misma estructura, facilitando el intercambio y procesamiento de datos.

Ejemplo de DTD para una Receta

Imagina que quieres asegurarte de que cada receta XML tenga siempre un nombre, un tiempo de preparación y al menos un ingrediente. La DTD lo garantiza:

```
<!-- Declaración de la DTD -->
<!DOCTYPE receta [
  <!-- El elemento raíz 'receta' contiene nombre, tiempo e
  ingredientes -->
  <!ELEMENT receta (nombre, tiempo, ingredientes)>
  <!-- 'nombre' y 'tiempo' solo contienen texto -->
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT tiempo (#PCDATA)>
  <!-- 'ingredientes' contiene uno o más 'item' -->
  <!ELEMENT ingredientes (item+)>
  <!-- 'item' solo contiene texto -->
  <!ELEMENT item (#PCDATA)>
]>
```

Con esta DTD, cualquier "receta" que no incluya un `<nombre>`, `<tiempo>` o `<ingredientes>`, o que los contenga en un orden incorrecto, será considerada inválida.

XSD: Esquemas XML Más Potentes

XSD (XML Schema Definition) define las **reglas que un documento XML debe seguir**: qué elementos existen, qué contienen y en qué orden aparecen. Es más poderoso y preciso que DTD.



xs:element

¿Qué elemento hay? Declara un elemento XML, su nombre y el tipo de dato que almacena.

```
<xs:element name="precio" type="xs:decimal"/>
```



xs:complexType

¿Qué contiene? Se usa cuando un elemento no tiene texto simple, sino que **contiene otros elementos** dentro.

```
<xs:complexType>
  ...elementos hijos...
</xs:complexType>
```



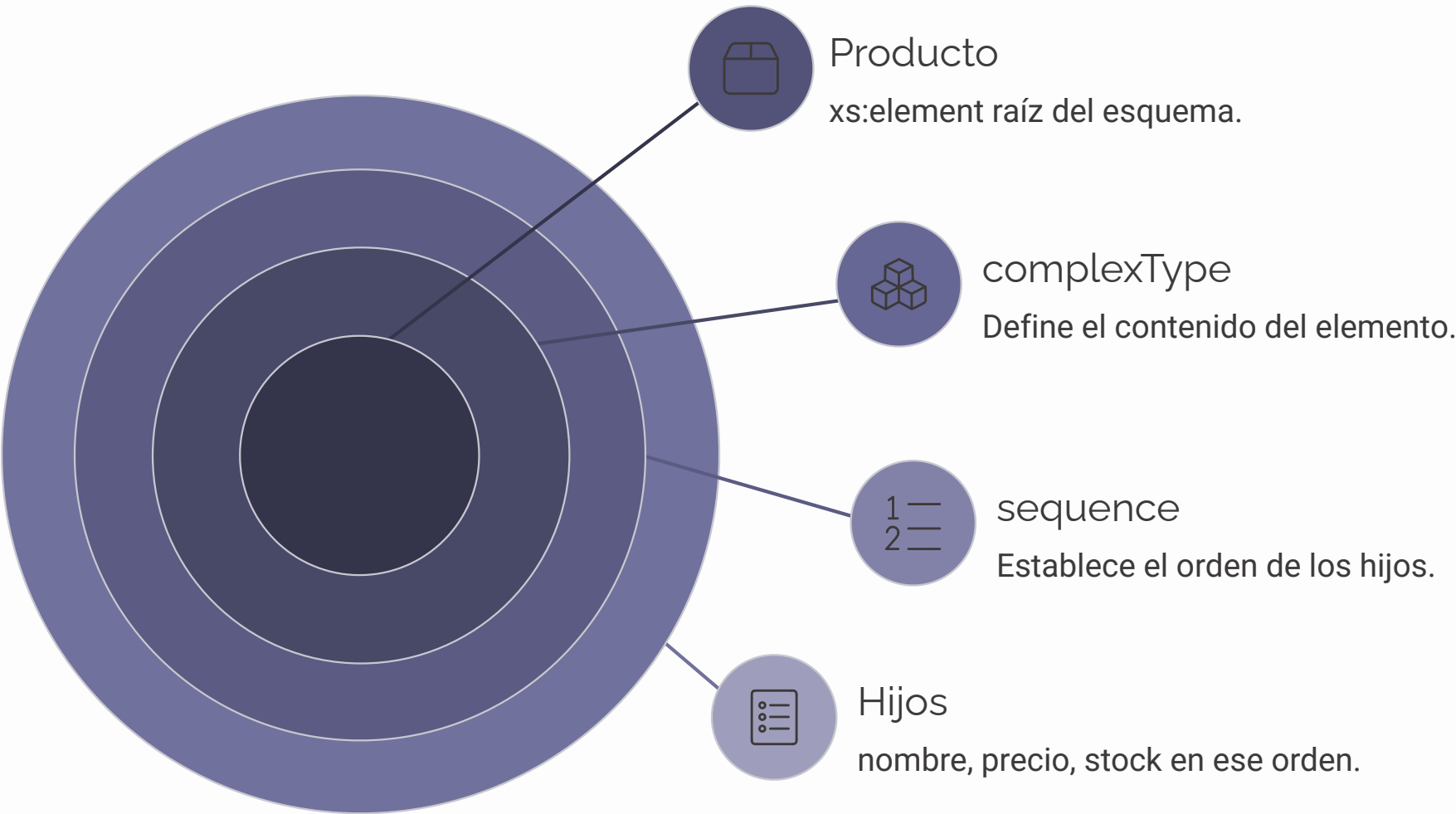
xs:sequence

¿En qué orden? Indica que los elementos hijos deben aparecer en un **orden específico y obligatorio**.

```
<xs:sequence>
  ...en este orden exacto...
</xs:sequence>
```

Jerarquía en Acción: Elemento <producto>

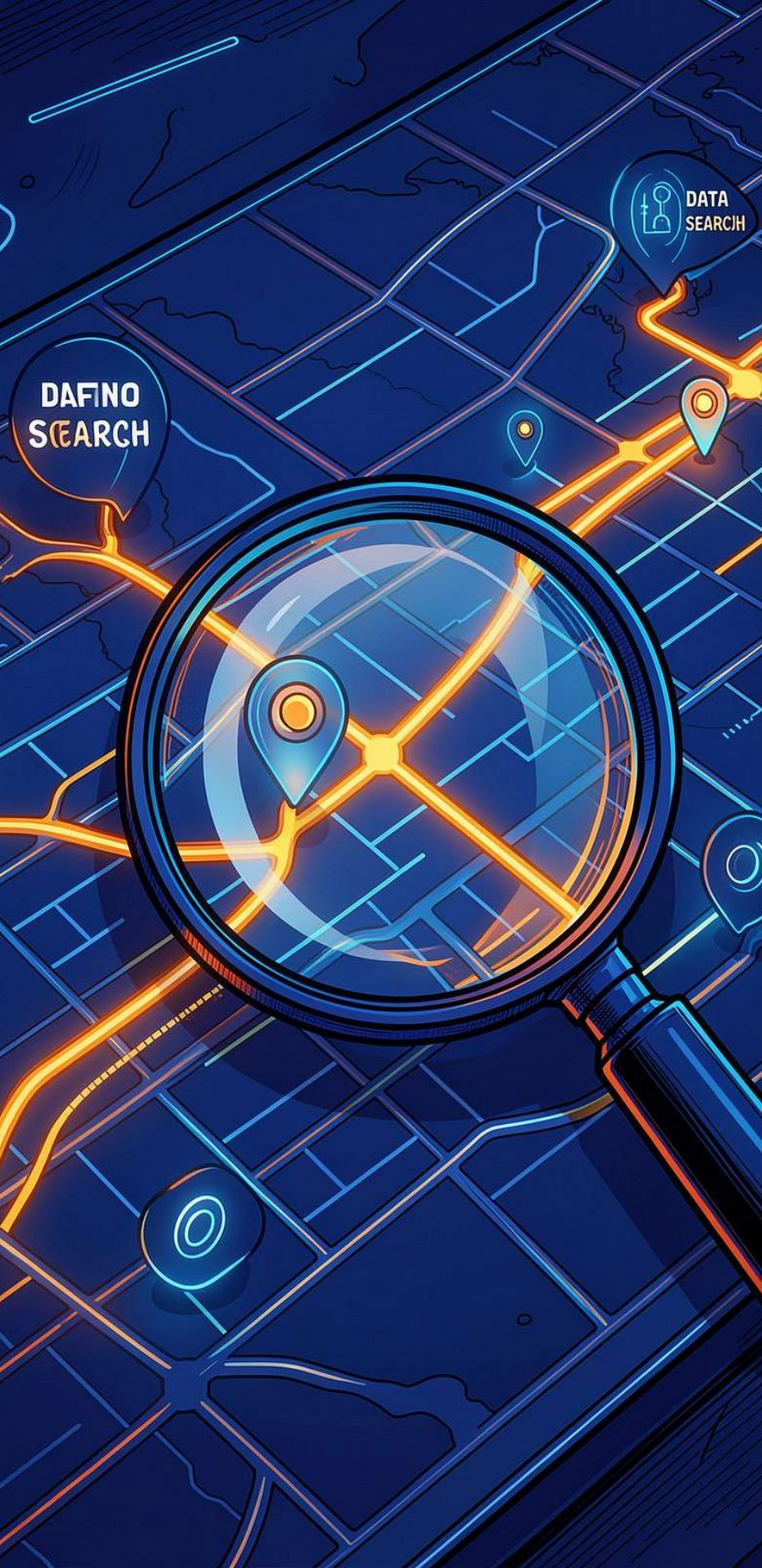
Así se combinan los tres conceptos para definir un elemento complejo con estructura ordenada:





xs:attribute → Información adicional del elemento

Un atributo añade datos extra al elemento, como un identificador. Por ejemplo: `<xs:attribute name="id" type="xs:string" use="required"/>` obliga a que cada `<producto>` tenga un atributo `id`, como `id="P001"`.



XPath: Navegar y Buscar en XML

XPath (XML Path Language) es un lenguaje diseñado para seleccionar nodos o partes específicas dentro de un documento XML. Piensa en él como un "mapa" para encontrar datos valiosos.

Función Clave

Permite navegar la estructura de un XML para extraer datos de forma eficiente.

Selección Precisa

Puedes seleccionar elementos, atributos o texto basándote en su ubicación o contenido.

Ejemplos Básicos

Para seleccionar todos los elementos `<ingrediente>` en cualquier parte del documento:

```
//ingrediente
```

Para encontrar el `<nombre>` de la `<receta>` (ruta absoluta):

```
/receta/nombre
```

XPath es fundamental para la manipulación y transformación de datos XML en aplicaciones, por ejemplo, en la web para extraer información de páginas (web scraping) o para la configuración de software.